

```

import threading
import time
import random

class TokenRing:
    def __init__(self, num_processes):
        self.num_processes = num_processes
        self.token = True # Token initially with process 0
        self.current_holder = 0
        self.lock = threading.Lock()
        self.running = True

    def critical_section(self, process_id):
        print(f"Process {process_id} ENTERING critical section.")
        time.sleep(random.uniform(1, 2)) # Simulate work
        print(f"Process {process_id} EXITING critical section.")

    def request_token(self, process_id):
        while self.running:
            with self.lock:
                if self.current_holder == process_id and self.token:
                    self.critical_section(process_id)
                    self.token = False # Release token
                    self.current_holder = (process_id + 1) % self.num_processes
                    self.token = True
            time.sleep(1)

    def start(self):
        threads = []
        for i in range(self.num_processes):
            t = threading.Thread(target=self.request_token, args=(i,))
            threads.append(t)
            t.start()

        try:
            while True:
                time.sleep(0.1)
        except KeyboardInterrupt:
            self.running = False
            print("\nStopping simulation...")
            for t in threads:
                t.join()

if __name__ == "__main__":
    ring = TokenRing(num_processes=5)
    ring.start()

```

